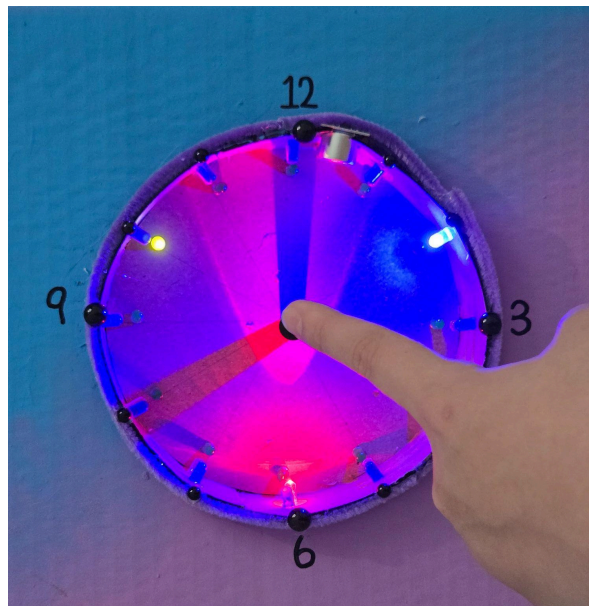


The ShadowPlay Clock



Chloe Pang and Alary Sivaprakash

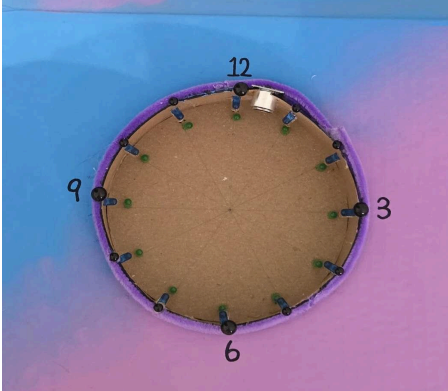
TEJ4M1-02

June 12th, 2026

Table of Contents

Device Overview.....	3
Safety info.....	3
User instructions.....	3
Research.....	3
Parts Needed.....	3
Circuit Diagram.....	4
Code Breakdown.....	4
Reflection.....	8
References.....	8

Device Overview

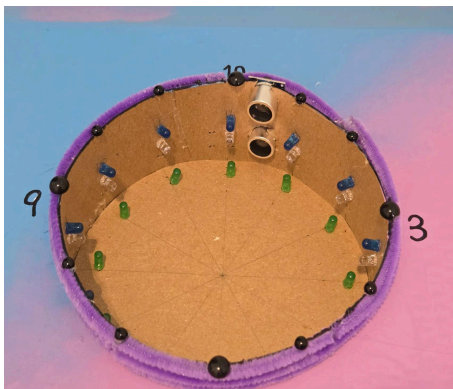


The ShadowPlay Clock is a device that displays the current time at the touch of your finger. It is a cross between a sundial and a modern analog clock, using light and shadow to mimic the hour and minute hands.

The clock has a motion sensor that is activated when a finger touches the center of the clock. Once triggered, LEDs surrounding the clock turn on and off corresponding to the current time. The shadow from the blue light represents the hour hand; the shadow from the red light represents the minute hand; the flashing green light represents the second hand.

Because the shadows come from different coloured light, the hour and minute hands are easily distinguishable using the RGB additive colour model. The clock face appears purple because of the combination of red and blue light. The hour hand appears red, since the blue light is removed. The minute hand appears blue, since the red light is removed.

 Shadow Clock Demo.mp4



Safety info

This device uses low voltage electricity and has minimal exposed wires since it is stored in a cardboard box. However, to ensure maximal safety, follow the safety guidelines below:

- Keep food and drink away from the power supply and arduino.
- Do not place the device on wet surfaces.
- Avoid touching any exposed wires.

- Unplug the arduino and turn off the power supply when not in use.
- When in use, ensure that the power supply does not exceed 5 volts.
- Verify all wiring connections before powering the circuit to prevent short circuits.

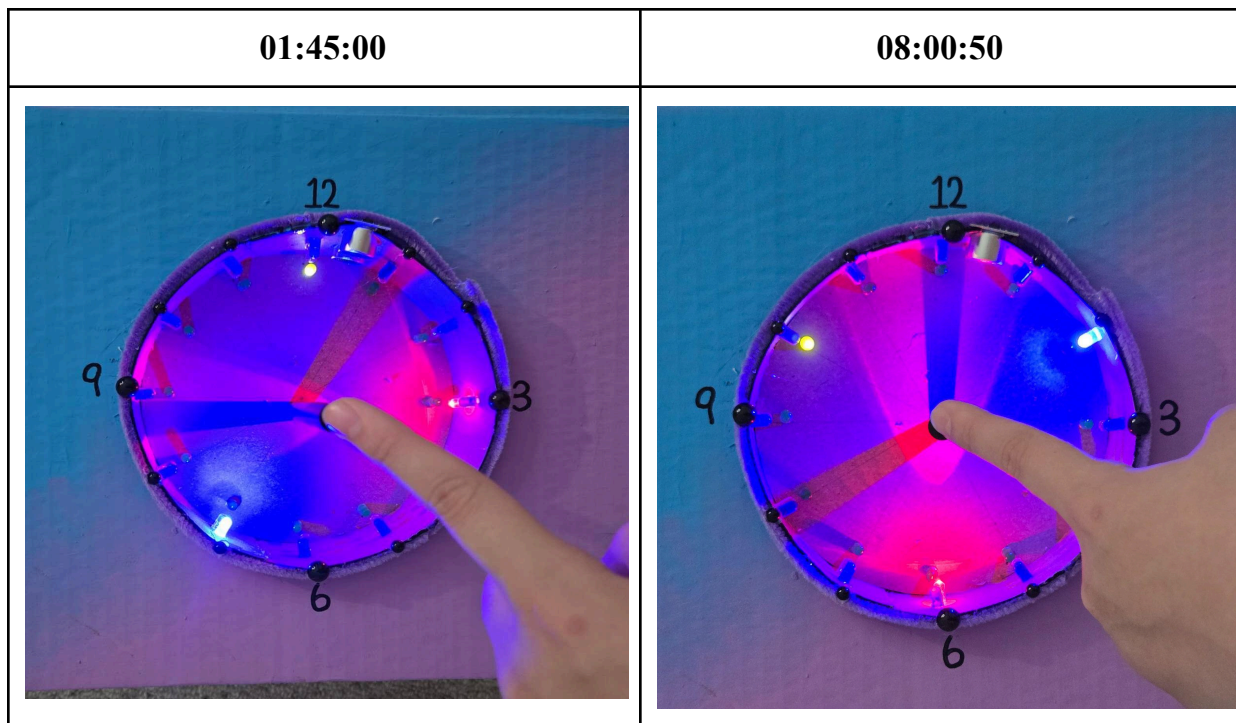
- Do not force components into the breadboard, as this may damage the pins or create loose connections.
- LEDs may become slightly warm after extended operation; allow components to cool before handling if necessary.
- Keep the device away from small children, as jumper wires and electronic components may present a choking hazard.
- Operate the device in a well-lit, dry indoor environment to reduce the risk of accidental damage.

- Avoid bending or pulling wires excessively, as damaged wires may cause unreliable operation.
- Disconnect power immediately if the user notices unusual heat, smoke, a burning smell, or malfunctioning components

User instructions

The ShadowPlay Clock uses two breadboards, one arduino UNO, as well as an additional power supply. Assemble the circuit diagram provided (refer to page 11) using the parts listed below (refer to page 9). Ensure that the power supply is connected to the breadboard once the circuit is assembled, and connect the arduino to your computer using the USB 2.0 cord. Use Arduino IDE to upload the given code for the clock (refer to page 14). To calibrate the clock, add the current local time in the designated parts of the code.

Once assembled and programmed, the user will be able to read the current time at the touch of a finger. Touch your outstretched finger to the center of the clock, perpendicular to the clock face. Alternatively, the user may use a pencil or pen. The ultrasonic sensor will detect your finger if it is in the center of the clock face, and LEDs corresponding to the current hour, minute, and second will light up, similar to a standard analog clock. The finger's shadow from the blue light will point to the exact hour. The finger's shadow from the red light will point to the minute rounded to the lowest multiple of five. The green light will point to the second rounded to the lowest multiple of five. See the examples below.



Research

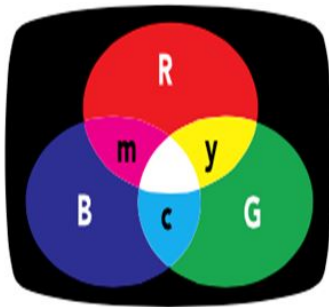
Inspiration

We were inspired by the early concept of sundials, using the sun's light to cast different shadows at different times of day. Combining the idea of light and shadows with modern, analog clocks would allow them to read the time more readily and accurately than a sundial.

RGB Additive Colour Model

To differentiate between the different hands on a clock, we decided to use red, blue, and green light using the RGB additive colour model.

- This colour model starts in darkness (black) and *adds* red, green and blue light to produce the visible spectrum of colors. The most dominant regions of colour in the visible spectrum are red, green, and blue, the additive primary colours.
 - The three main light receptors in the retinas of our eyes perceive short (blue-ish), middle (green-ish), and long (red-ish) wavelengths of light. A mixture of these receptors are used to see the full range of colors in the light spectrum.
- As shown in the image below, red and blue light combine to make magenta; red and green light combine to make yellow; and green and blue light combine to make cyan.



Using this colour model, we know that if all of the coloured LEDs are lit up, we will have white light. Placing your finger in the shadow removes one of the primary and changes the colour. For example, the shadow from the blue light will only have red and green light, making the shadow appear yellow. We can use this concept to make the hands on the clock different colours.

Brainstorming

Clock calculations

For the hour display, the 24-hour value is first converted to a 12-hour format. This allows times like 13:00 and 1:00 to be displayed in the same position. The clock then determines which of the 12 positions represents the current hour.

The minute display is divided into 12 equal parts. Since an hour has 60 minutes, each position represents a 5-minute interval. For example:

- Position 0 represents 0 minutes
- Position 1 represents 5 minutes
- Position 2 represents 10 minutes
- Position 3 represents 15 minutes

(This pattern continues until position 11, representing 55 minutes.)

The second display follows the same principle. Since one minute has 60 seconds, and there are 12 positions on the dial, each position represents a 5-second interval. Therefore, the second hand jumps once every 5 seconds, not once per second.

For example, at 1:45:30:

- Hour position = 1 $\rightarrow$ 7 o'clock position (Opposite direction to create shadow)
- Minute position = $45 \div 5 = 9 \rightarrow$ Position 3 (Opposite direction to create shadow)
- Second position = $30 \div 5 = 6 \rightarrow$ Position 6 (Opposite direction to create shadow)

Formula:

Hour Position = $(\text{Hour} \bmod 12 + 6) \bmod 12$

The additional 6-position offset is required because the LEDs are placed opposite to the shadow direction. Turning on the opposite LED causes the shadow to appear at the correct clock position.

Ex:

$(1+6) \bmod 12 = 7$

Therefore, at 1:00 the LED at position 7 is activated, producing a shadow that points to the 1 o'clock position.

Formula:

Minute Position = $((\text{min}/5) + 6) \bmod 12$

Ex:

$$45 \div 5 = 9$$

$$(9+6) \bmod 12 = 3$$

Position 3 creates a shadow at the 45-minute mark.

Formula:

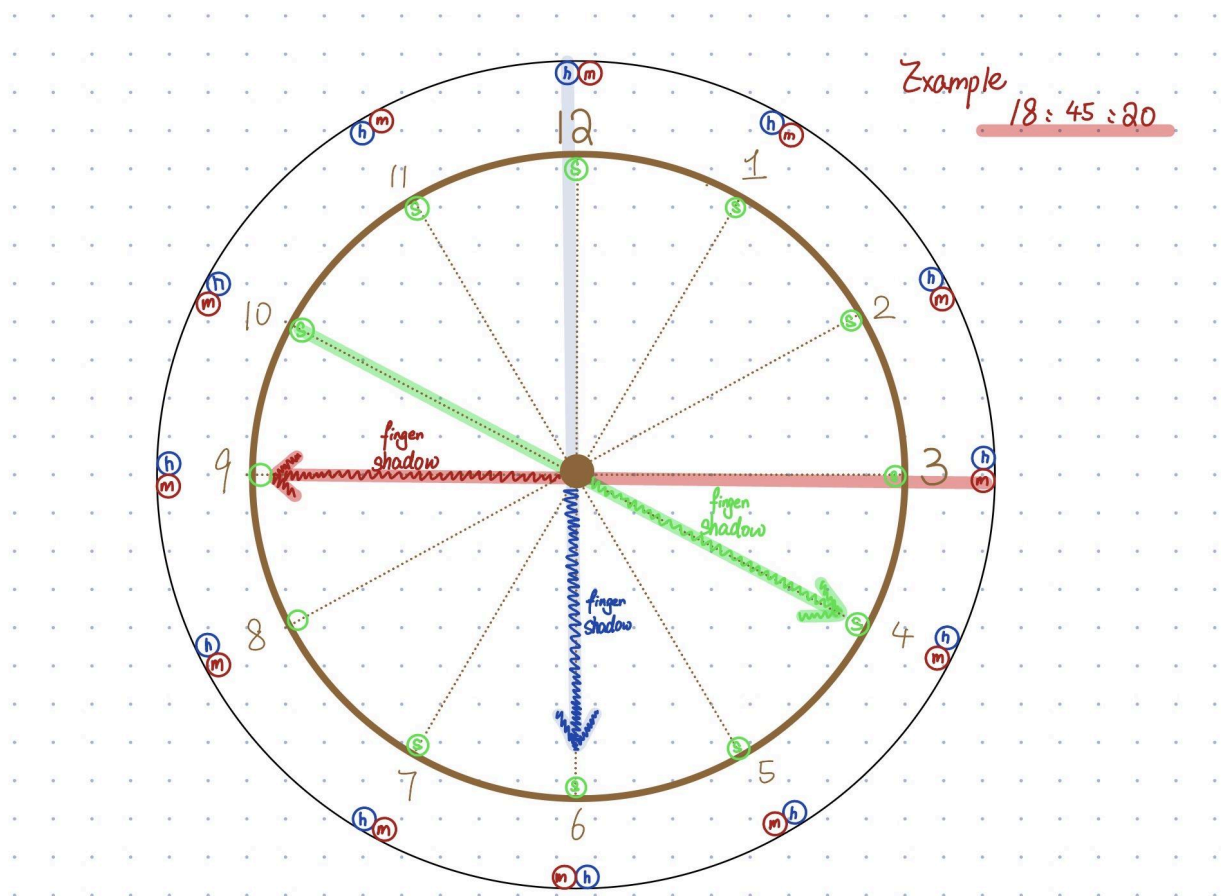
$$\text{Second Position} = (\text{sec}/5) \bmod 12$$

Ex:

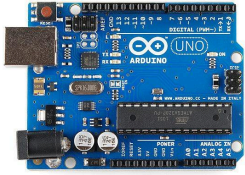
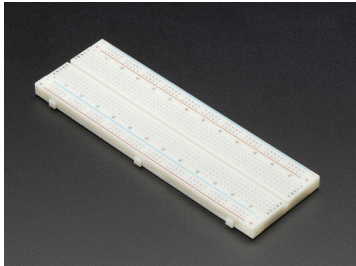
$$30 \div 5 = 6$$

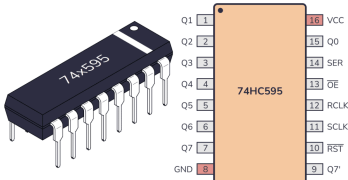
The LED at position 6 turns on, representing 30 seconds.

Sketches



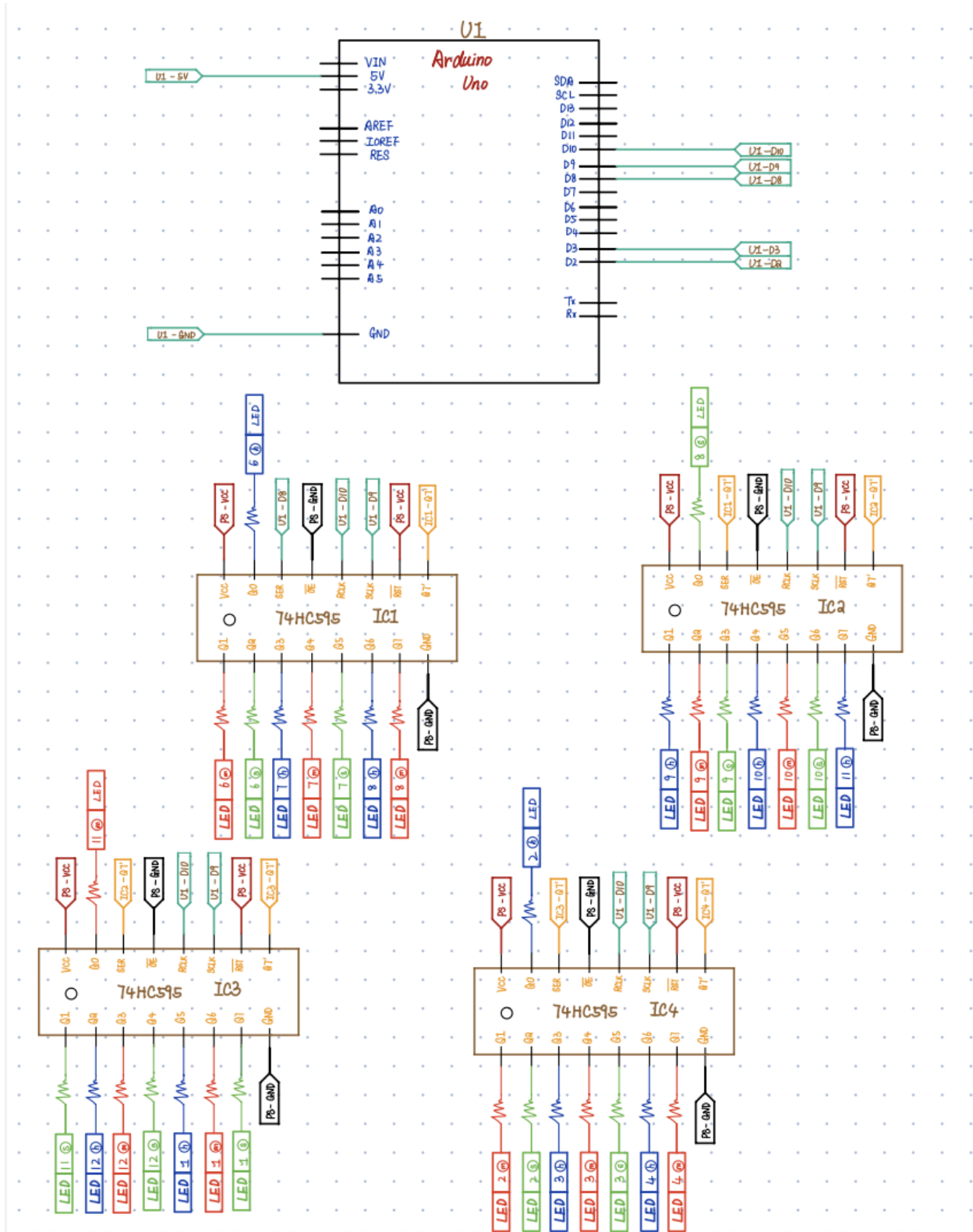
Parts Needed

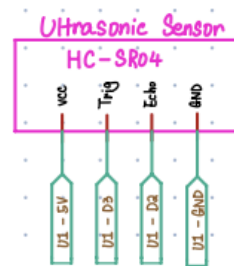
Part	# of parts	Image
Arduino UNO	1	
USB 2.0 cord	1	
Breadboard	2	
Blue LEDs	12	
Green LEDs	12	
LCDs (red)	12	
220 ohm Resistor	36	

Ultrasonic Sensor	1	
Male-male jumper wires	Several	
Female-male jumper wires	Several	
74HC595	5	
Power supply	1	

Circuit Diagram

PDF Circuit Diagram.pdf





Code Breakdown

```
// Shadowplay Clock – Soft clock (not using RTC)
// The second hand jumps once every 5 seconds (0, 5, 10, ..., 55)
// The three lights display the "current time" only when the distance is approximately 7
cm ( $\pm 1$  cm); otherwise, they are off.

// Define pins used
#define TRIG 2
#define ECHO 3

#define DATA 8
#define CLOCK 9
#define LATCH 10

#define LEDPIN 13

//LED buffer
byte leds[5] = {0, 0, 0, 0, 0}; // 5字节=40位, 实际用36位

// Soft Clock
//Time can be modified, h/m/s
int h = 1; //hour
int m = 45; //min
int s = 0; //sec
unsigned long lastTickMs = 0;

// Motion sensor control
unsigned long readPulse()
{
    digitalWrite(TRIG, LOW);
    delayMicroseconds(5);
    digitalWrite(TRIG, HIGH);
    digitalWrite(LEDPIN, HIGH);
    delayMicroseconds(12);
    digitalWrite(TRIG, LOW);
```

```

digitalWrite(LEDPIN, LOW);
return pulseIn(ECHO, HIGH, 60000UL); //60 ms timeout
}

//Distance should be 7cm;No echo should returned 1000
long readDistanceOnce()
{
    unsigned long t = readPulse();
    if (t == 0) return 1000;
    return (long)(t * 0.0343 / 2.0);
}

// Take the median of the third iteration to obtain the steady state
long readDistanceMed3()
{
    unsigned long t1 = readPulse();
    unsigned long t2 = readPulse();
    unsigned long t3 = readPulse();
    unsigned long lo = min(t1, min(t2, t3));
    unsigned long hi = max(t1, max(t2, t3));
    unsigned long med = t1 + t2 + t3 - lo - hi;
    if (med == 0) return 1000;
    return (long)(med * 0.0343 / 2.0);
}

// 74HC595 helpers * 5
void writeLEDs()
{
    digitalWrite(LATCH, LOW);
    for (int i = 4; i >= 0; i--)
    {
        shiftOut(DATA, CLOCK, MSBFIRST, leds[i]);
    }
    digitalWrite(LATCH, HIGH);
}

void setLED(int index, bool on)

```

```

{
    int chip = index / 8;
    int bit  = index % 8;
    if (on)  leds[chip] |= (1 << bit);
    else     leds[chip] &= ~(1 << bit);
}

void alloFF() { for (int i = 0; i < 36; i++) setLED(i, false); }
void alloN()  { for (int i = 0; i < 36; i++) setLED(i, true);  }

// Show clock (12x3, Physical offset +6)
//index * 3 + 0 = hour level, +1 = minute level, +2 = second level
void showClock(int hh, int mm, int ss)
{
    alloFF();

    // Hours: 12-hour format, maintaining original +6 offset.
    int hourPos = (hh % 12 + 6) % 12;

    // Minutes: One unit every 5 minutes (0, 5, 10, ..., 55) plus a +6 offset.
    int minPos  = ((mm / 5) + 6) % 12;

    // Second hand: Jumps once every 5 seconds
    int secPos  = ((ss / 5) % 12);
    setLED(hourPos * 3 + 0, true); // hour
    setLED(minPos  * 3 + 1, true); // min
    setLED(secPos  * 3 + 2, true); // every 5 sec
}

//Soft clock tick
void tickClock()
{
    unsigned long now = millis();
    if (now - lastTickMs >= 1000UL)
    {
        unsigned long elapsed = (now - lastTickMs) / 1000UL; // full sec
        lastTickMs += elapsed * 1000UL;
    }
}

```

```

    s += (int)elapsed;

    if (s >= 60) { m += s / 60; s %= 60; }

    if (m >= 60) { h += m / 60; m %= 60; }

    if (h >= 24) { h %= 24; }

}

}

//Arduino setup/loop
void setup() {
    Serial.begin(9600);

    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    pinMode(LEDPIN, OUTPUT);
    digitalWrite(TRIG, LOW);
    digitalWrite(LEDPIN, LOW);

    pinMode(DATA, OUTPUT);
    pinMode(CLOCK, OUTPUT);
    pinMode(LATCH, OUTPUT);

    for (int i = 0; i < 5; i++) leds[i] = 0;
    writeLEDs();

    lastTickMs = millis();
}

void loop()
{
    tickClock();

    // Read distance
    // Use the median value for stability first
    long d = readDistanceMed3();

    //shows when  $\approx 7$  cm
    const float targetCm = 7.0;

```

```

const float tol = 1.0;

bool active = (d > 0) && (d >= targetCm - tol) && (d <= targetCm + tol);

// Testing output
Serial.print("time=");

if (h < 10) Serial.print('0'); Serial.print(h);
Serial.print(':');
if (m < 10) Serial.print('0'); Serial.print(m);
Serial.print(':');
if (s < 10) Serial.print('0'); Serial.print(s);
Serial.print("  dist=");
Serial.print(d);
Serial.print("  cm  active=");
Serial.println(active ? "YES" : "NO");

digitalWrite(LEDPIN, active ? HIGH : LOW);

// Display time only when active, otherwise turn off.
if (active) showClock(h, m, s);
else      alloFF();

writeLEDs();
delay(50);
}

```

Evaluation/Reflection

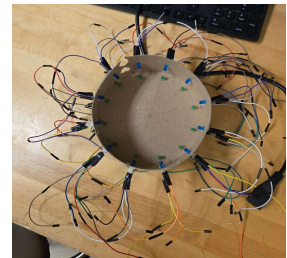
Goals Achieved

Our goal was to create a functioning clock that displays the current time when touched. The final product was very accurate to our initial sketches and designs.

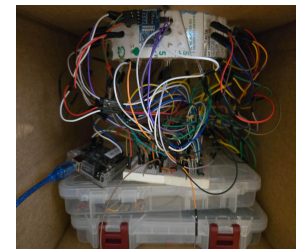
- We successfully calibrated the 36 LEDs to correspond with the preset time
- We successfully positioned the 36 LEDs at the correct positions depending on the clock hand (hour, minute, second hand; 12, 1, 3 o'clock, etc)
- We successfully implemented a sensor to detect a finger at a designated distance away, and successfully programmed the lights to turn on when the sensor is triggered.

Goals Missed

Initially, we had planned on representing the second hand with a shadow from the green light, and using a real-time clock to calibrate the current time that would be displayed on the device.



- The real-time clock did not work after a lot of troubleshooting, so we integrated code using formulas in order to manually input the current time.
- Because we had 36 LEDs and an ultrasonic motion sensor, we had upwards of 76 wire connections between the two breadboards, the arduino, and the device. This caused the testing and troubleshooting process to be longer than expected.



Areas of Improvement

Overall, the ShadowPlay Clock was very successful. However, improvements we could have made include the coding/building process as well as hardware organization.

- If the project were redesigned, we would improve wire management by using shorter colour-coded wires and creating a custom PCB.
- We would also add a real-time clock (RTC) module so the device automatically keeps accurate time without requiring manual calibration.

Self-Evaluation

- The ShadowPlay clock successfully displays the current time using colored shadows, showcasing concepts from electronics, programming, physics, and color theory.
- Despite the challenges of wiring and testing, the final device operates reliably and achieves all of our initial design goals.
- As a group, we were very organized and efficient, dividing the work evenly to ensure that we met the required deadlines.

References

Additive versus Subtractive Color Models | X-Rite Blog. (2018). X-Rite.

<http://xrite.com/blog/additive-subtractive-color-models>

Additive & Subtractive Color Models. (2024, October 1). DINFOS Pavilion.

<https://pavilion.dinfos.edu/Article/Article/2355687/additive-subtractive-color-models/>